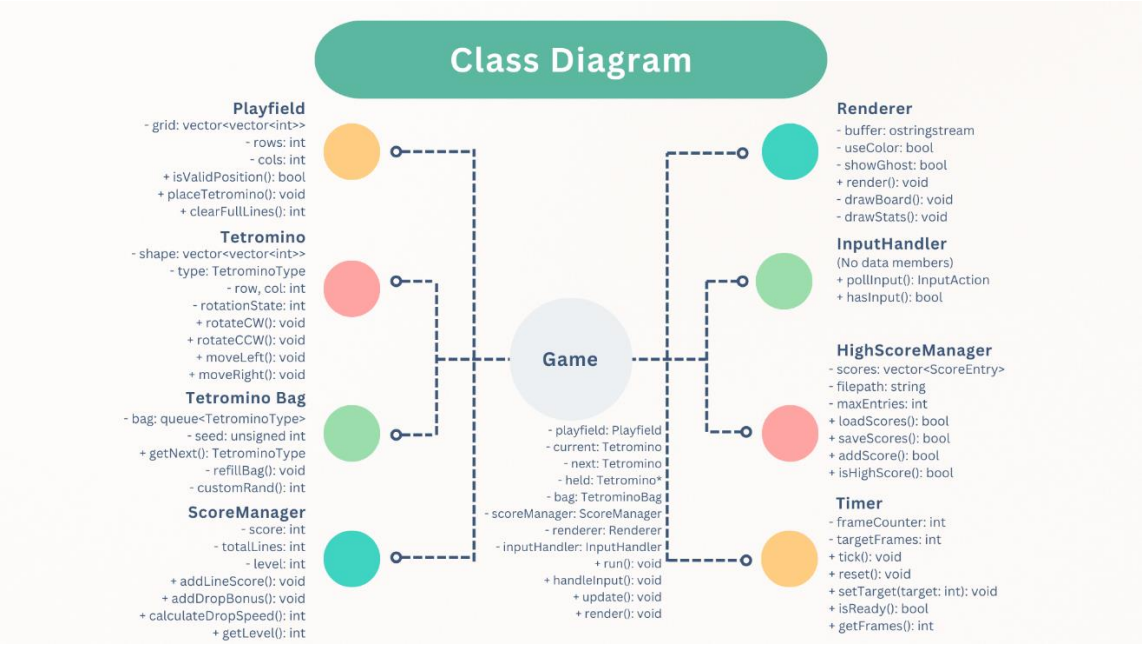
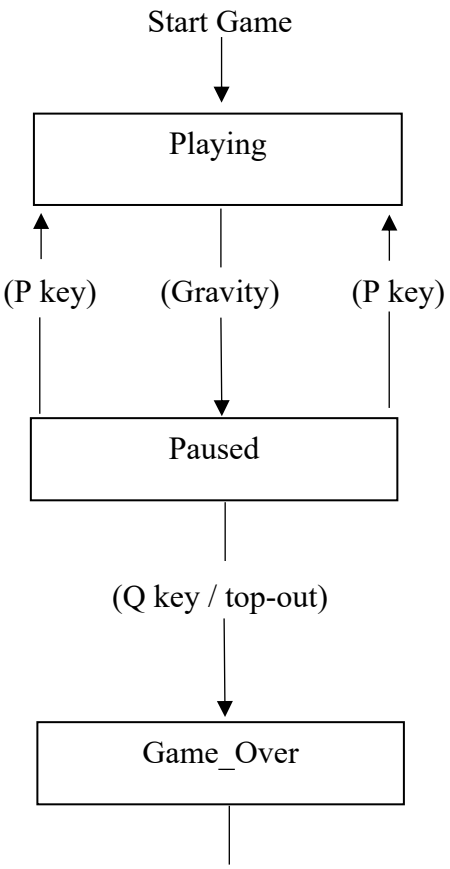
Tetris Design Brief

1.1 Class/Module Diagram



1.2 State Machine Diagram



High Score? If Yes, Prompt Name and Save Score. If No, Display Final Score

2.1 System Invariants

1. **Playfield Integrity:** All cells are -1 (empty) or 0-6 (piece type); no overlaps
2. **Active Piece Validity:** Current piece never overlaps locked cells or exceeds bounds
3. **Line Clear Consistency:** After clear, no gaps in rows; all rows above shift down
4. **RNG Determinism:** Same seed → identical piece sequence via LCG
5. **Level Bounds**: $1 \leq \text{level} \leq 10$ (enforced by std::min())
6. **Hold Restriction:** Can hold once per piece drop (reset after lock)

2.2 Rotation System

Method: 90° clockwise matrix transpose

Formula: rotated[j][n-1-i] = original[i][j]

States: 0, 1, 2, 3 (O-piece: always 0)

State 0: State 1: State 2: State 3:

[1] [1] [1][1][1] [1]

[1][1][1] [1][1] [1] [1][1]

[1] [1]

CCW: Apply CW 3 times or use inverse formula

Wall Kicks: 7 test offsets for normal pieces (9 for I-piece)

- (0,0) original, 2. (0,-1) left, 3. (0,+1) right, 4. (-1,0) up, 5. (+1,0) down, 6-7. Extended kicks

2.3 Collision Detection

Method: Check all 4 blocks of piece against playfield

```
for each block in piece.shape:
  newRow = piece.row + block.row + deltaRow
  newCol = piece.col + block.col + deltaCol
  if (out of bounds OR cell occupied): return INVALID
return VALID
```

Complexity: O(4) = O(1)

2.4 Bag Randomization

- Algorithm: Fisher-Yates shuffle of all 7 pieces
- Guarantee: All 7 types appear once before any repeat
- Max drought: 13 pieces (2 bags - 1)
- Determinism: LCG with seed: seed = (seed × 1103515245 + 12345) & 0x7fffffff

3.1 Complexity Summary Table

Operation	Data Structure	Time Complexity	Space Complexity
Cell Access	2D Vector	O(1)	O(R×C)
Collision Check	Shape Matrix	O(4) = O(1)	O(16)
Line Detection	Row Scan	O(C) per row	-
Line Clear	Full Scan + Compact	O(RxC)	-
Rotation	Matrix Transpose	$O(k^2) \approx O(1)$	O(16)
Next Piece	Queue (bag)	Amortized O(1)	O(7)
Bag Refill	Fisher-Yates	$O(7 \log 7) \approx O(1)$	O(7)
Score Update	Variables	O(1)	O(1)
Rendering	Full Redraw	O(RxC)	O(R×C)
Ghost Piece	Simulate Drop	O(R)	O(1)
High Score Add	Vector + Sort	O(n log n), n≤10	O(10)

Where: R=rows (20), C=cols (10), k=piece dimension (≤4), n=scores (≤10)

4. Rendering Strategy

Method: Buffered output with ANSI cursor repositioning

1. Build entire frame in std::ostringstream buffer
2. Use ANSI \033[H to move cursor to home (no clear)
3. Output buffer in single write: std::cout << buffer.str()
4. Flush immediately

Flicker Prevention:

- No system("cls") between frames
- Single atomic write operation
- Cursor hidden during gameplay
- 20 FPS stable performance

5. Configuration System

Method: JSON file + CLI override
Priority: CLI args > config file > hardcoded defaults
Format: Nested JSON with validation on load
Configurable:

- Board size (10-30 rows, 8-20 cols)
- Starting level (1-10)
- Seed (0 = random)
- Drop speeds, scoring values
- Controls (key bindings)
- Display (colors, ghost piece)

6. Known Limitations

- Platform: Windows-only (<conio.h>, <windows.h>)
- Lock Delay: Basic (no extended delay or infinity)
- Wall Kicks: Simplified (not full SRS specification)
- Next Preview: Single piece (not multiple)

7. Testing Strategy

Unit Tests: Collision, rotation, line clear algorithms
Integration: Full game loop with deterministic seeds
Boundary: Min/max board sizes, wall rotations, stack heights
Deterministic Verification:

- Same seed produces identical piece sequences
- Logged in tests/seeded_run_*.log files
- Confirmed through 3 independent test runs